ALOY: A Local-First AI Operating System with Deterministic Agent Control, Persistent Semantic Memory, and Policy-Gated Tool Execution

Author:	Dhanush A.
Document:	Technical Report & Systems Architecture Specification
Version:	1.0 Release Candidate
Prepared For:	AI Safety Fellowship -- Systems Engineering Track
Date:	July 2026

Abstract

This paper describes the architecture and implementation of ALOY, a locally-deployed AI operating system designed to coordinate autonomous agents with persistent memory and bounded tool execution. ALOY runs all model inference on local hardware via Ollama, stores state in a single SQLite database with FTS5 and sqlite-vec extensions, and enforces execution boundaries through filesystem sandboxing, policy-based permission evaluation, and human-in-the-loop confirmation gates. The system addresses four problems common to cloud-hosted agent frameworks: (1) data leaves the local environment during inference, (2) conversational state is lost between sessions, (3) agents can execute arbitrary system operations without oversight, and (4) small local models lose coherence when context windows overflow. We describe each subsystem, discuss engineering trade-offs, present the threat model, and examine relevance to practical AI safety engineering.

Table of Contents

1. Introduction

Agent systems built on large language models present a fundamental tension: they require broad system access to be useful, but that access creates safety risks that grow with autonomy. A coding agent that can read files, run shell commands, and edit source code is far more capable than a stateless chatbot -- but it can also delete directories, exfiltrate data, or enter infinite execution loops if left unconstrained.

Cloud-hosted agent frameworks compound these risks. User data -- source files, database contents, conversation histories -- must traverse network boundaries to reach inference servers. Session state is ephemeral; when a conversation ends, all accumulated context disappears. And tool execution typically occurs without structured human oversight.

ALOY is a local-first AI operating system designed to address these problems. It runs all inference on local hardware using Ollama-managed models, maintains persistent semantic memory in a local SQLite database, enforces agent execution boundaries through a finite state machine with workspace snapshots, and gates destructive tool operations through a policy engine that requires explicit user confirmation.

1.1 Problem Scope

Data boundary violation. Standard agent frameworks forward local files and queries to external APIs. For proprietary codebases, medical records, or corporate data, this creates unacceptable privacy exposure. ALOY keeps all data on the local machine.

Session state loss. Chatbot architectures are stateless across sessions. When a session terminates, accumulated context is discarded. ALOY maintains a four-tier persistent memory system that survives across sessions and consolidates semantically over time.

Unconstrained tool execution. Giving agents access to shell environments and file systems creates destructive potential. ALOY interposes a sandbox validator and policy engine between the agent and the operating system, requiring human confirmation for write and execute operations.

Context window overflow. Local models typically have 4K-16K token context windows. ALOY implements a token budget manager that allocates fixed budgets per context section and prunes content that exceeds its allocation.

1.2 Scope and Limitations

ALOY is a systems engineering project, not a research contribution to alignment theory. It does not solve the alignment problem, produce novel training methods, or advance interpretability research. Its contribution is architectural: it demonstrates concrete engineering patterns -- FSM-based agent control, transactional workspace recovery, policy-gated execution, and persistent semantic memory -- that reduce the operational risk of deploying local autonomous agents.

2. Design Principles

Seven principles guided ALOY's architecture:

- Local-first containment. All model inference, embedding generation, and tool executions occur on local hardware. No data leaves the machine during normal operation.
- Modular subsystem isolation. The codebase separates concerns into independent modules (conversation, memory, models, agent, tools, security, database) communicating through defined interfaces.
- Transactional execution. Before an agent executes a task step, the system creates a workspace snapshot. If the task fails, the AdvancedRollbackEngine restores the workspace to its pre-execution state.
- Deterministic state transitions. Agent sessions are governed by a finite state machine with seven states and explicitly enumerated transitions. The FSM rejects invalid transitions at runtime.
- Defense-in-depth safety. Tool operations pass through three sequential gates: sandbox path validation, policy engine evaluation, and human confirmation via SSE.
- Tiered memory with decay. Memory is organized into four tiers with configurable decay rates and promotion thresholds. A background ConsolidationEngine periodically promotes, archives, and deduplicates memories.
- Asynchronous pipeline composition. The conversation engine processes messages through five sequential pipeline stages. Adding a new step requires only implementing the PipelineStage interface.

3. System Architecture

ALOY consists of six principal subsystems connected through a FastAPI backend. Figure 1 shows the high-level component layout.

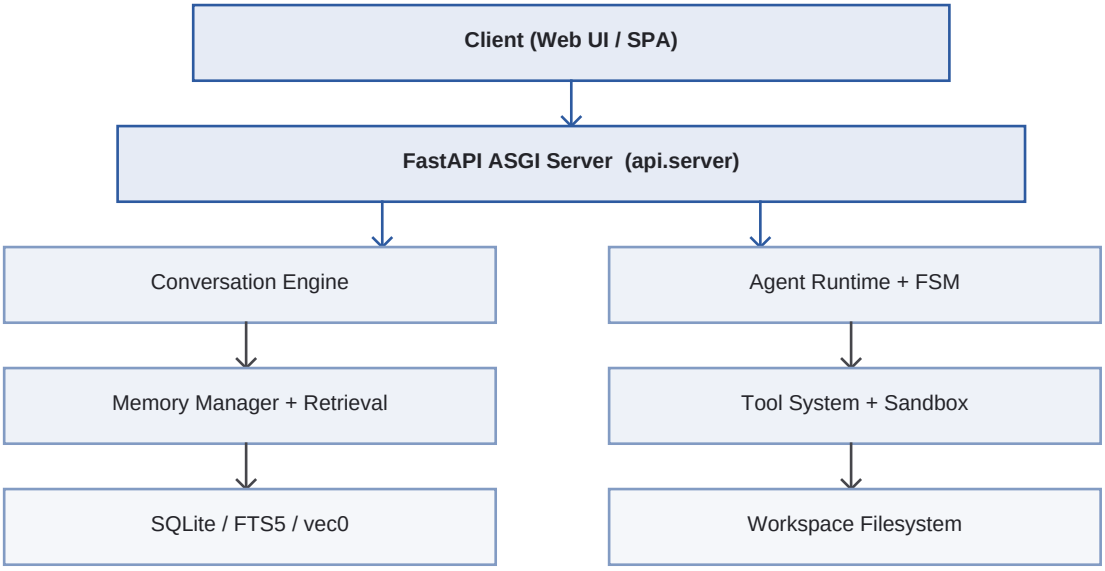

Figure 1: Modular layered architecture of ALOY. Arrows indicate primary data flow.

3.1 Component Responsibilities

The FastAPI server manages the application lifecycle: it initializes the database connection pool, runs schema migrations, instantiates all subsystem coordinators, and exposes 10 REST routers. Response tokens are delivered via Server-Sent Events. On startup, the server creates a running.tmp lockfile; if the server crashes, the file persists and is detected on the next boot for crash recovery.

The conversation engine orchestrates user interactions through five pipeline stages. The memory manager provides storage, retrieval, and consolidation services. The agent runtime coordinates multi-step autonomous tasks through FSM-governed sessions. The tool system resolves tool metadata, validates sandbox boundaries, requests authorization, and executes tools with timeout control.

4. Conversation Pipeline

The conversation engine implements a five-stage pipeline that transforms a user message into a streamed model response. Each stage is a class inheriting from PipelineStage with a single async method: process(context) -> context.

Figure 2: Five-stage conversation processing pipeline.

4.1 Intent Detection

The IntentDetectionStage classifies messages into one of eight intent categories: simple_chat, complex_chat, coding_request, reasoning_request, planning_request, memory_query, tool_request, and meta_request.

Classification uses a two-phase approach. First, five sets of regular expressions run against the lowercased input. If a pattern matches, the intent is returned immediately without invoking a model. If no pattern matches, the stage falls back to LLM-based classification: it constructs a structured prompt listing all valid intents and asks the model (at temperature 0.0) to output exactly one. If the LLM call fails, the stage defaults to simple_chat.

4.2 History Load and Memory Retrieval

The HistoryLoadStage retrieves the most recent 10 conversation turns from SQLite. The MemoryRetrievalStage queries the memory subsystem for up to 20 candidate records relevant to the current message, using the hybrid FTS5 + vector retrieval described in Section 5.

4.3 Context Assembly

The ContextBuildStage assembles the final prompt within a configurable token budget (default: 8000 tokens). It constructs a date-aware system prompt, resolves the active identity context from the IdentityEngine, and calls the ContextIntelligenceEngine to produce a ContextPack -- a token-budgeted combination of system prompt, identity text, selected memories, and pruned history.

The stage also evaluates whether the query requires live web data. A _needs_live_search function checks for freshness keywords (51 terms including 'news', 'weather', 'latest', 'price', 'stock') or year patterns matching 2024-2099. If triggered, the stage executes a web search, parses results, deduplicates by URL, scores source credibility against a whitelist of 17 high-trust domains, and injects curated results as a structured XML block.

4.4 Token Budget Management

The TokenBudgetManager counts tokens using tiktoken (cl100k_base encoding) when available, falling back to a 4-character-per-token estimate. It allocates budgets per intent type -- for example, reasoning_request reserves 6000 tokens for model generation while limiting memories to 1000 tokens, whereas memory_query allocates 3000 tokens to memories and only 500 to history.

5. Memory Subsystem

ALOY implements a hierarchical memory system organized into four tiers, each with distinct retention characteristics.

Tier	Persistence	Decay	Protection
Working	In-process RAM	Discarded on session end	None
Short-term	SQLite	decay_rate = 0.01	None
Long-term	SQLite	Reduced rate	Promotion at 5 accesses
Permanent	SQLite	No decay	is_protected = 1

Table 1: Memory tier characteristics.

5.1 Hybrid Retrieval: FTS5 + Vector Similarity

Memory retrieval combines two complementary search mechanisms. Full-text search uses an FTS5 virtual table (memories_fts) configured with Porter stemming and Unicode61 tokenization, indexing the content, summary, and category columns. Vector similarity search uses a vec0 virtual table (memory_embeddings) storing 768-dimensional float32 embeddings generated by nomic-embed-text.

The RetrievalEngine executes both searches (each returning up to 50 candidates), then merges results using Reciprocal Rank Fusion (RRF) with k=60. The final hybrid score combines three signals: 50% normalized RRF rank, 25% importance score, and 25% recency (computed as exp(-0.02 * hours_since_access)).

5.2 Consolidation Engine

The ConsolidationEngine runs during idle periods and performs four maintenance operations in sequence.

Step 1 -- Promotion. Queries for short-term memories with access_count >= 5 and updates their tier to long_term.

Step 2 -- Archival. Identifies non-protected memories with importance below 0.05, compresses them, and marks them as archived.

Step 3 -- Deduplication. Loads up to 500 active memories with embeddings and identifies pairs with cosine similarity >= 0.92. For each pair, a six-factor merge score is computed:

```
MergeScore = 0.35 * CosineSim
           + 0.20 * (1.0 - |imp_a - imp_b|)
           + 0.15 * mean(confidence_a, confidence_b)
           + 0.15 * mean(recency_a, recency_b)
           + 0.10 * TagOverlap
           + 0.05 * GraphLinkWeight
```

If the merge score exceeds the dedup_threshold (0.80), the less important memory is merged into the more important one.

Step 4 -- Clustering. Runs informational semantic clustering over up to 300 active memories, producing between 2 and 10 clusters for diagnostic logging.

5.3 Memory Decay

Importance scores decay exponentially: I_new = I_old exp(-lambda t), where lambda is the per-memory decay_rate (default 0.01) and t is idle time in hours. The DecayEngine applies a 5% significance threshold -- updates smaller than 0.05 are skipped to reduce write pressure.

6. Model Routing

ALOY routes inference requests through a centralized ModelRouter that maps 16 task types to specific local models. The router serves as the single gateway for all LLM calls -- no subsystem accesses the Ollama client directly.

Task	Primary	Fallback	Priority
simple_chat	phi4-mini	qwen3:14b	CONVERSATION
coding_request	qwen2.5-coder:14b	qwen3:14b	CONVERSATION
reasoning	deepseek-r1:14b	qwen3:14b	CONVERSATION
agent_planning	qwen3:14b	qwen2.5-coder:14b	AGENT
agent_coding	qwen2.5-coder:14b	qwen3:14b	AGENT
memory_gen	phi4-mini	qwen3:14b	BACKGROUND
embedding	nomic-embed-text	(none)	BACKGROUND

Table 2: Model routing configuration (selected entries).

The priority system controls scheduling through a ModelCoordinator with a semaphore (max_concurrent=1). CONVERSATION-priority requests preempt BACKGROUND tasks. When the primary model fails, the router catches the exception, records it via ModelTracker, and retries with the fallback. The router also maintains conversation continuity bindings: once a model is used for a conversation, subsequent messages route to the same model.

7. Agent Runtime & Finite State Machine

The agent runtime coordinates multi-step autonomous task execution. It is the most safety-critical subsystem in ALOY, as it governs how much autonomy an agent has and under what conditions that autonomy can be revoked.

7.1 Session Lifecycle

An agent session begins when the runtime creates a new session record with status PLANNING. The session is associated with a project, a goal, and an ExecutionContext carrying workspace metadata, cancellation tokens, allowed tools, and paths. Execution spawns an asyncio task running the ManagerAgent, which decomposes the goal into steps and enqueues them. Worker agents claim tasks and execute them using the tool system.

7.2 Finite State Machine

The AgentSessionFSM enforces seven states with explicitly enumerated valid transitions:

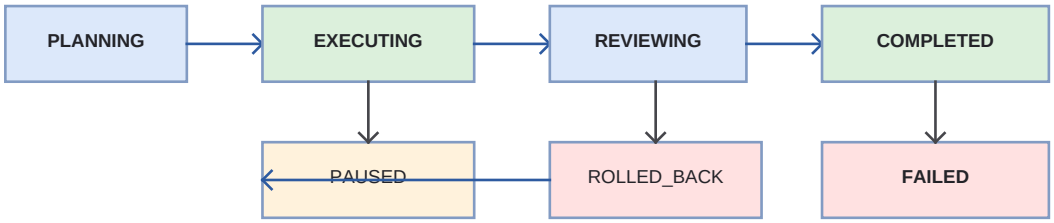

Figure 3: Agent session FSM with seven states. COMPLETED is terminal.

The transition_to method reads the current state from the database, checks whether the target state is in the allowed set, and raises InvalidStateTransitionError if not. COMPLETED is terminal: no further transitions are possible. FAILED allows only ROLLED_BACK, and ROLLED_BACK allows transitions back to PLANNING or EXECUTING.

7.3 Workspace Snapshots

Before executing a task step, the WorkspaceSnapshotManager creates a recoverable snapshot. In git repositories, it creates a branch (aloy-snap-{session}-{timestamp}), commits all changes, and switches back. Otherwise, it creates a zip archive excluding .git, __pycache__, .aloy, node_modules, and venv directories.

7.4 Rollback

The AdvancedRollbackEngine supports full workspace rollback (git reset --hard or zip extraction) and single-file rollback (git checkout of individual files or zip extraction of specific entries). The single-file rollback includes a path traversal guard: it calls target.relative_to(workspace_path) and raises PermissionError if the target resolves outside the workspace boundary.

8. Tool Execution & Safety Gating

The tool system manages the lifecycle of tool invocations, from registry lookup through sandbox validation, authorization, execution, and result formatting.

8.1 Registered Tools

The system registers 12 tools at startup: FileEditor, Terminal, PythonRunner, Git, Browser, WebSearch, PDFReader, ImageReader, SQLite, Docker, Runner, and DiffEngine. Each tool declares its required permissions and timeout in its ToolMetadata.

8.2 Execution Flow

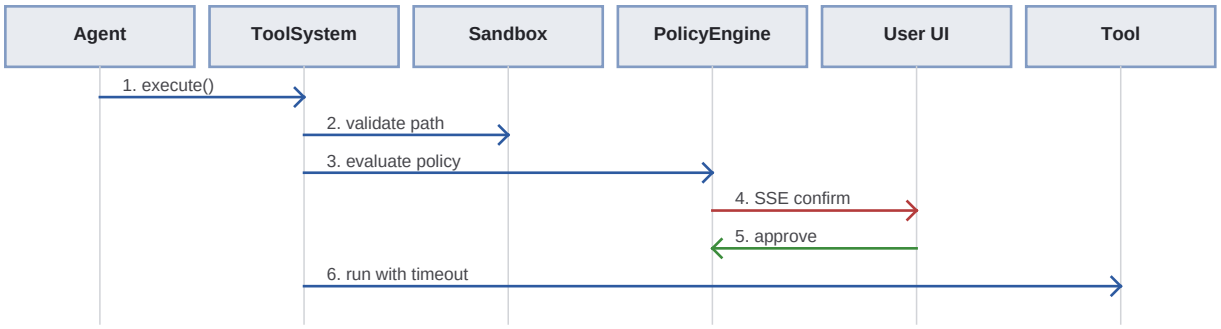

Figure 4: Tool execution sequence with three safety gates.

The system applies dynamic permission specialization: if a file_editor tool is invoked with action=read, the required permission is downgraded from write_file to read_file. Similarly, git status, git diff, and git log are treated as read-only. This reduces confirmation fatigue while maintaining safety for actual modifications.

8.3 Policy Engine

The PolicyEngine classifies actions by string prefix: read/list/view/search are approved automatically; write/delete/modify/execute require approval with auto_approve_for_session=True (subsequent writes in the same session are auto-approved after the first). Unknown action types are denied by default.

9. Database Layer

ALOY uses a single SQLite database file (data/aloy.db) for all persistent state.

9.1 Connection Architecture

The DatabaseConnectionPool manages a single serialized write connection (protected by threading.Lock with auto-commit/rollback) and a pool of up to 5 read-only connections opened with ?mode=ro. If sqlite-vec is installed, it is loaded into every connection via sqlite_vec.load(). All connections use WAL mode, PRAGMA synchronous = NORMAL, and a 64MB page cache.

9.2 Core Schema

```
CREATE TABLE memories (
    id TEXT PRIMARY KEY,
    type TEXT NOT NULL,
    tier TEXT NOT NULL DEFAULT 'short_term',
    content TEXT NOT NULL,
    confidence REAL DEFAULT 0.5,
    importance REAL DEFAULT 0.5,
    access_count INTEGER DEFAULT 0,
    decay_rate REAL DEFAULT 0.01,
    is_protected INTEGER DEFAULT 0,
    embedding BLOB
);

CREATE VIRTUAL TABLE memory_embeddings
    USING vec0(embedding float[768]);

CREATE VIRTUAL TABLE memories_fts
    USING fts5(content, summary, category,
    content='memories', tokenize='porter unicode61');
```

10. Validation Framework

The validation framework (validation.mega_audit) provides automated quality assurance. It generates 150+ test prompts across 40+ categories including identity integrity, version reporting, creator attribution, privacy, security, memory honesty, web search, hallucination detection, code generation, reasoning, and prompt injection resistance.

Each response is scored across 18 quality dimensions (identity, memory, accuracy, naturalness, hallucination, logic, reasoning, formatting, routing, latency, privacy, safety, consistency, context, search, friendliness, professionalism, human_feeling). Deductions are specific: identity scores lose 40 points if the response omits 'ALOY', 50 points if a competitor model name appears; latency scores are penalized at 1000ms, 2000ms, and 5000ms thresholds.

Results are aggregated into a release readiness score combining pass rate (40%), critical bug count (30%), identity failures (20%), and hallucination risks (10%).

11. Threat Model & Security Boundaries

ALOY assumes the local host environment is trusted but treats all model inputs, executed scripts, and web search results as potentially adversarial.

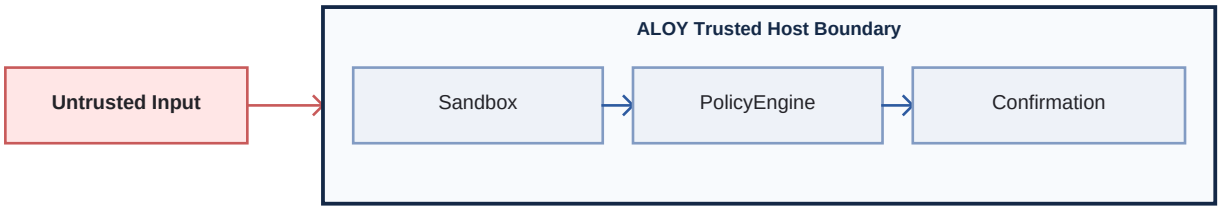

Figure 5: Three-gate defense-in-depth within the trusted host boundary.

11.1 Threat Vectors

Prompt injection: even if the model follows an injected instruction, the sandbox blocks paths outside the workspace, the policy engine requires approval for writes, and the confirmation gate pauses for human review.

Path traversal: Sandbox.is_within_workspace resolves all paths including symlinks via Path.resolve() and verifies descendance from a registered workspace root.

Resource exhaustion: tool executions are wrapped in asyncio.wait_for with per-tool timeouts. The ModelCoordinator limits concurrent inference to prevent OOM.

11.2 Residual Risks

The sandbox validates paths at the Python API level but does not monitor kernel-level file I/O. Tools using os.symlink or ctypes could bypass the check. The policy engine uses string-prefix matching, which could be bypassed by mislabeled action types. Automated test environments that auto-approve confirmations lose the human safety layer.

12. Engineering Trade-offs

Trade-off	Benefit	Cost
Local inference	Data privacy, no network dependency	Lower reasoning quality
SQLite + vec0	Zero-dependency, single-file DB	Linear scan, no ANN
Three-gate safety	Defense-in-depth	Confirmation friction
Context pruning	Prevents overflow	May lose old context
Single-GPU model lock	Prevents OOM	Background tasks blocked

Table 3: Principal engineering trade-offs in ALOY.

Each trade-off reflects a deliberate design decision optimized for single-user, local-first deployment on consumer hardware. Different deployment contexts -- multi-user servers, cloud environments, enterprise settings -- would shift these trade-offs toward different solutions.

13. Limitations & Known Issues

- Intent classification: regex patterns miss edge cases; LLM fallback adds 1-3s latency.
- Single-GPU contention: user chat, embedding generation, and agent tasks compete for hardware; the coordinator serializes access (max_concurrent=1).
- Rollback audit trail: the engine restores files but does not generate file-level diffs showing what changed.
- Single-user design: SQLite lacks user-level access control; shared instances have no memory or permission isolation.
- Token counting fallback: when tiktoken is unavailable, the 4-char/token estimate can miscount, risking context overflow or underutilization.

14. Relevance to AI Safety

ALOY does not solve the alignment problem. It does not produce new training methods, interpretability tools, or theoretical safety guarantees. Its contribution is operational: it demonstrates engineering patterns that reduce the practical risk of deploying autonomous agents on local systems.

- Bounded autonomy: the FSM prevents uncontrolled execution loops. Every session has a well-defined lifecycle, and the system enforces that transitions follow declared rules.
- Transactional recoverability: workspace snapshots ensure that any task step can be fully undone. Destructive actions are always recoverable.
- Human oversight gates: no write or execute operation proceeds without explicit human approval (or session-level pre-approval).
- Memory integrity: consolidation prevents memory bloat from degrading retrieval quality. Decay ensures irrelevant memories are gradually archived.
- Containment boundaries: the sandbox ensures operations cannot escape the workspace. Combined with the policy engine's default-deny posture, this creates a containment boundary around the agent's operational scope.

These mechanisms do not guarantee safety in the formal sense -- they can be circumvented by sufficiently adversarial inputs or implementation bugs. But they raise the engineering bar: a failure must bypass multiple independent checks to cause harm, which is substantially harder than bypassing a single-layer defense.

15. Future Work

- Formal FSM verification using model checking (TLA+, Spin) to prove the state machine cannot reach unhandled states.
- Uncertainty estimation by monitoring token probability distributions to detect low-confidence tool commands.
- Kernel-level sandboxing (seccomp, Windows sandbox APIs) to prevent tools from bypassing Python's path APIs.
- Distributed validation across multiple processes for larger regression test suites.
- Multi-user isolation with user-scoped memory partitioning and per-user approval policies.

16. Conclusion

ALOY demonstrates that a locally-deployed AI agent system can provide persistent memory, multi-model task routing, and autonomous code execution while maintaining meaningful safety boundaries. The architecture -- a five-stage conversation pipeline, four-tier memory hierarchy with semantic consolidation, seven-state FSM for agent control, and three-gate tool execution safety system -- addresses the core operational risks of deploying autonomous agents: data leakage, state loss, unconstrained execution, and context overflow.

The system makes explicit trade-offs between capability and safety: local inference reduces model quality but eliminates data exposure; workspace snapshots add storage overhead but enable full recoverability; confirmation gates introduce friction but prevent unauthorized operations. These are engineering decisions that reflect the reality that deployed AI systems must balance performance against operational risk.

The codebase provides a concrete reference implementation for researchers and engineers interested in building safe, local-first agent systems -- not as a solved problem, but as a practical starting point for systems that take agent safety seriously.

ALOY v1.0 -- Technical Fellowship Work Sample Submission

Author: Dhanush A. | July 2026